Multilayered Fault Detection and Localization With Transformer for Microservice Systems

Jingyu Wang, *Senior Member, IEEE*, Yuewei Li, Qi Qi, *Senior Member, IEEE*, Yan Lu, and Bo Wu

***Abstract*—Software architecture is undergoing a transition from monolithic architecture to microservices to achieve resilience, agility, and scalability in the software life cycle. The complex dependability of these microservices may lead to unexpected failures, which becomes a major concern on reliability for application providers. The existing fault detection and localization algorithms for microservice systems only focus on the relationship within microservices and cannot achieve finer granularity from a layered system perspective, including microservices, containers, physical machines, and networks. To tackle this problem, we propose a multilayered method that deconstructs cloud-based microservices and connects the information from various layers to enhance the precision of fault detection and localization. The proposed Transformer encoder model can detect anomalies of containers in the resource layer, and by decomposing and analyzing invocation latency, anomalies in the service layer can be detected. After determining the faulty area of the resource layer based on the above anomalies, a multifactor root cause score is used to sort root cause metrics in the faulty area for localization. Evaluations were performed on three datasets: the Sock-Shop dataset we collected from an actual microservice system, the AIOps2020 preliminary dataset, and the SMD. Empirical investigations conducted on these datasets show that our models enhance the F1 score by approximately 0.25 for anomaly detection and improve the mean average precision by up to 0.54 for root cause localization, which underscores the utility of our models in effectively managing microservice systems in practical scenarios.**

***Index Terms*—Anomaly detection, artificial intelligence for IT operations, fault localization, microservice systems, transformer.**

NOMENCLATURE

Notation	Description
N	Number of containers in the microservices system.
M	Number of metrics collected from each container.
T	Duration of metrics collection (in minutes).
$x^t_{(n,m)}$	Value of metric m collected from container n at timestamp t.
$x^t_{(n,m)}$	Reconstructed value of metric m collected from container n at timestamp t.
$\mathcal{T}$	Collection of metrics after collecting T minutes for all containers.
X	Time-series data partitioned into time windows for input.
Y	Prediction results for input time window data.
$\mathcal{X}$	Input data for temporal attention module.
D	Dimension of the rows of matrix $\mathcal{X}$.
H	Number of heads in multihead self-attention.
W	Weights of fully connected network.
Q	Query matrix in transformer.
K	Key matrix in transformer.
V	Value matrix in transformer.
Z	Input for positionwise feed-forward module.
O	Output of temporal block.
R	Input for container block
A	Correlation matrix of attention.
s	Anomaly score.

Manuscript received 19 April 2023; revised 21 August 2023; accepted 16 December 2023. Date of publication 6 February 2024; date of current version 4 September 2024. This work was supported in part by the National Natural Science Foundation of China under Grant U23B2001, Grant 62171057, Grant 62101064, Grant 62201072, Grant 62001054, and Grant 62071067, in part by the National Postdoctoral Program for Innovative Talents under Grant BX20230052, in part by China Postdoctoral Science Foundation under Grant2023TQ0039, in part by the Ministry of Education and China Mobile Joint Fund under GrantMCM20200202 and Grant MCM20180101, and in part by theBeijing University of Posts and Telecommunications-China Mobile Research Institute Joint Innovation Center. Associate Editor: A. Hamou-Lhadj. *(Corresponding author: Qi Qi.)*

Jingyu Wang, Yuewei Li, Qi Qi, and Yan Lu are with the State Key Laboratory of Networking, Switching Technology, Beijing University of Posts, Telecommunications, Beijing 100088, China (e-mail: wangjingyu@bupt.edu.cn; liyuewei2000@bupt.edu.cn; qiqi8266@bupt.edu.cn; luyanfcp@bupt.edu.cn).

Bo Wu is with the China Mobile Research Institute, Beijing 100053, China (e-mail: wubo@chinamobile.com).

Digital Object Identifier 10.1109/TR.2024.3356717

I. INTRODUCTION

WITH the continuous expansion of business, the limitations of traditional monolithic architecture have become increasingly apparent in meeting the demands of modern applications. Monolithic applications are typically constructed from a single codebase that packages all functions together. Consequently, they are unable to perform partial expansion, increasing the system's throughput and capacity. The microservice architecture [1] splits a monolithic application into multiple small autonomous services. These services can be scaled and updated independently, leading to improved flexibility, scalability, and maintainability of the system. Container [2], [3] is a virtualization technology that can package applications and all their dependencies together to form an independent, portable, and deployable unit. Compared to virtual machines, it has more relaxed isolation properties to share the operating system among the applications. Therefore, containers are considered lightweight. Container technology accelerates the process of microservices development, deployment, and delivery, while also providing a unified solution for diverse deployment environments. The microservice architecture, which divides the system into multiple independent modules with loose coupling, has

1558-1721 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

many benefits. However, it also brings about service dependence characteristics that may lead to unexpected errors and raises concerns about the reliability of microservices. As microservice systems become more complex, the probability of problems arising also increases. Failure to promptly detect and locate these issues can compromise the reliability and stability of the system, ultimately affecting the user experience and business efficiency.

Unlike other scenarios, such as anomaly detection in time-series data and fault localization at the level of program statements [4], maintaining reliability with microservices presents us with the following four challenges. 1) Distributed and heterogeneous systems: Typically, microservice systems run on multiple machines, and different services may perform differently in terms of metrics due to factors such as development languages. 2) Big data: In the industry, there can be hundreds of or thousands of microservices, and it is easy to achieve millions of data to deal with. 3) Dynamic dependencies: The interconnections among containerized services are dictated by the constantly evolving business logic. The constituent containers within services dynamically scale and migrate in response to load variations and the state of the underlying machines. 4) Fault propagation [5]: Due to the complex dependability of services, faults can quickly spread between services, making it difficult for operators to locate the root cause. To cope with these challenges, so far there have been many studies on system anomaly detection and root cause localization. Concerning anomaly detection, several classic outlier detection methods, such as local outlier factor (LOF) [6] and support vector machines (SVMs) [7], continue to be applied to the anomaly detection of microservice system metrics. However, these methods are primarily utilized for swiftly identifying obvious abrupt anomalies and are challenged by accurately handling intricate intramicroservice data patterns and dependencies. Deep learning techniques [8], [9] have also been widely adopted for detecting anomalies in the multidimensional metrics of microservices. For instance, approaches like LSTM-AD [10] and TopoMAD [11] utilize long-short-term-memory (LSTM)-based models to capture temporal features for anomaly detection. However, recurrent models like LSTM are known for their slowness and substantial computational overhead. Thus, there is a demand for an efficient model capable of acquiring the complex features of microservices while avoiding these limitations. In the research on locating root causes, previous work mainly constructed a dependence graph [12] or a cause-effect graph [13] and performed root cause inference on the graph to locate root cause service. These existing methods do not pinpoint root causes at a finer granularity, offering limited assistance to operational personnel in identifying the fundamental reasons behind failures.

In order to conduct a more thorough analysis and address the limitations present in current methodologies, we deconstruct microservices into multiple layers and propose a more refined approach for anomaly detection and root cause localization. We decompose microservices into three layers: service layer, resource layer, and metric layer. The service layer refers to the running services, the resource layer encompasses the containers and physical machines supporting services, and the metric layer integrates essential metric data collected from resource layers. As depicted in Fig. 1(a), existing methods often focus solely on the service layer, and root cause localization often uses overly simplistic anomaly detection methods, which to some extent, can affect the accuracy of root cause localization. In contrast, as illustrated in Fig. 1(b), we integrate anomaly detection and root cause localization with finer granularity. Our approach initially identifies faulty areas within the resource layer through anomaly detection and then precisely traces back to the root cause metrics, which can better handle the complex scenario of cloud-based systems.

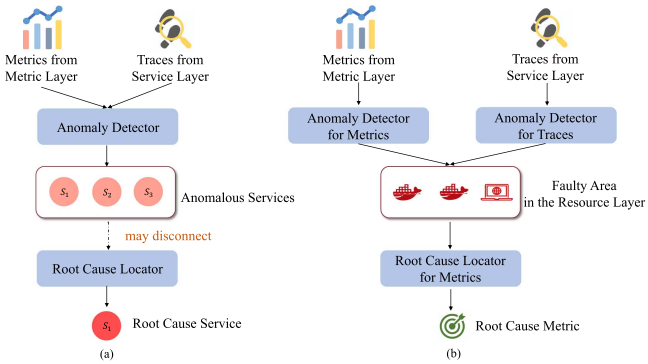


Fig. 1. (a) Existing scheme of fault detection and localization. (b) Proposed scheme of fault detection and localization.

Specifically, concerning the detection of anomalies in time-series metrics, we previously noted the inefficiency of recurrent models due to their sequential nature, where computations at each time step depend on the output of the previous one. The emergence of transformer [14] enables us to leverage attention mechanisms that simultaneously consider all inputs, facilitating a more comprehensive understanding of temporal features. Through GPU parallelism, transformer can detect anomalies faster than recurrent models. Consequently, we build a model based on the transformer architecture as the anomaly detector. We use it to not only learn temporal patterns but also capture dependencies between containers, enabling precise and efficient localization of anomalous containers. Building upon this foundation, we additionally employ breadth-first search (BFS) and kernel density estimation (KDE) for detecting invocation latency anomalies in traces. Coupled with metric analysis and ranking, this approach enables us to conclusively identify the root cause metrics responsible for the failure. The contributions of this article can be summarized as follows.

1) We propose a multilayered fault detection and localization method, which can detect faults in the resource layer and accurately locate them in the metric layer, significantly enhancing the system reliability and stability.
2) A transformer encoder model called TAD, designed for detecting anomalies at the container level in multivariate time-series data, is proposed. To accommodate the dynamic topology changes, we introduce the container block, which employs self-defined attention mechanisms.
3) We perform extensive experiments on three datasets to compare and analyze our models against baselines, and the results show that our models achieve improvement of up to about 0.25 in F1 score for detection and 0.54 in mean average precision for localization.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

4) We publish the Sock-Shop dataset we collected on GitHub[1] for better reproducibility of the results of this article.

The rest of this article is organized as follows. Section II introduces some related work. Section III overviews the fault detection and localization method we proposed. Section IV outlines the TAD model for anomaly detection. Sections V and VI present a faulty area discovery algorithm and root cause metric localization method. The evaluation and analysis of the proposed method are shown in Section VII. Finally, Section VIII concludes this article.

II. Related Work

A. Microservice Anomaly Detection

Existing efforts in microservices anomaly detection primarily revolve around monitoring metrics, logs [15], [16], or traces [17], [18]. Due to the easier analyzability and real-time nature of metrics, this subsection primarily focuses on presenting models for anomaly detection in microservice time-series data.

1) Classic Models: Classic methods often measure the degree of similarity between points and points by distances. Similar points are very close and points far away from most points are generally regarded as anomalies. LOF [6] calculates the density ratio between each data point and its neighboring data points to measure the outlierness of data points, where points with significantly different densities are identified as anomalies. On the other hand, one-class SVM [7] treats all normal data as a single class and employs support vector machines to create a boundary that separates normal data points from anomalies. These two algorithms are commonly employed in industry for quickly detecting evident anomalies. Therefore, we chose them as the baseline algorithms.

2) Forecasting-Based Models: The primary focus of the forecasting-based approach is to learn from normal data and predict future data points following the established normal pattern. As a result, anomalies can be detected by analyzing the prediction errors. LSTM-AD [10] constructs a stacked LSTM network to detect anomalies through predictive analysis. DAGMM [19] combines deep autoencoders and Gaussian mixture model (GMM) to forecast the next data point, enabling the detection of anomalies. It optimizes the parameters of two models simultaneously in an end-to-end fashion, which enhances its robustness and overall performance.

3) Reconstruction-Based Models: The main idea of these models is to reduce the original data dimension, map it to the latent space, and then reconstruct the original input. By learning only the normal data, the model can only reconstruct normal data and then evaluate the degree of anomaly through reconstruction errors. OmniAnomaly [20] constructs a stochastic recurrent neural network, which reconstructs input data from the representations and uses the reconstruction probabilities to determine anomalies. It demonstrates exceptional performance when applied to the dataset they collected within cloud environments. TopoMAD [11] combines graph neural networks, LSTM, and variational autoencoder to perform unsupervised anomaly detection for microservice systems. With the help of topological information, it can learn the spatial dependence of metrics.

All of these models except TopoMAD focus only on temporal relationships, and none of them achieve high efficiency and accuracy at the same time. In contrast to these models, our approach not only accounts for temporal relationships but also incorporates spatial relationships among containers. Leveraging the efficiency of the transformer architecture, our model efficiently detects anomalies.

B. Root Cause Localization

MonitorRank [21] takes into account both internal and external factors, constructs a call graph, and performs a random walk over this graph based on the similarity with frontend metrics. Microscope [22] employs the PC algorithm to construct a causal graph of services, where the subnodes of anomalous nodes are included in a candidate set. Nodes within this candidate set are evaluated for their correlation with the initial front-end anomalies using relevance calculations to determine the root cause. Using causal relationships to construct graph enhances the interpretability of this approach. MicroRCA [12] constructs a service dependence graph, leveraging random walks and similarity-based anomaly representation. Additionally, it takes into account the resource utilization of related components to represent service anomalies, which improves the accuracy of root cause localization.

While these methods all take similarity into account, metric similarity does not always effectively represent anomaly. In contrast, our proposed methodology involves a BFS traversal of the trace tree, employing statistical techniques for detecting anomalies in invocation latency. Moreover, our approach extends beyond mere service layer localization and delves further into the granular metric layer.

III. Method Overview

As shown in Fig. 2, we divide the microservice system into three layers: the service layer, the resource layer, and the metric layer. The service layer represents several running microservices, from which we can obtain information such as service dependence graph and traces. The resource layer refers to the underlying resources that support the operation of microservices, including containers and physical machines. We can gather information about the number and status of containers running on the machines. The metrics layer consists of various container and machine metrics collected from the resource layer. We can collect data from machines and containers through monitoring agents such as Node exportor[2] and cAdvisor,[3] and then Prometheus[4] periodically pulls metrics uniformly from the HTTP addresses exposed by these agents. These metrics are not only useful for anomaly detection and root cause analysis, but

[1][Online]. Available: https://github.com/OpenNetAI/Sock-Shop-Dataset

[2][Online]. Available: https://github.com/prometheus/node_exporter
[3][Online]. Available: https://github.com/google/cadvisor
[4][Online]. Available: https://prometheus.io

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

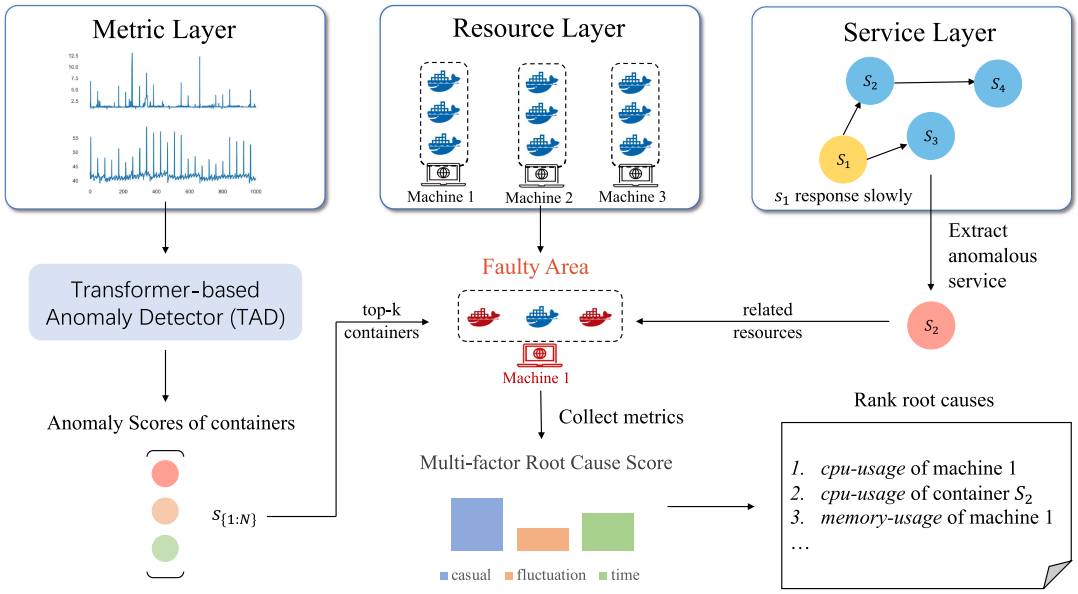


Fig. 2. Overview of proposed fault detection and localization method.

also for enhancing performance and resource planning, among other aspects.

We use the case of Fig. 2 to explain how the proposed approach works. Four services, S_1–S_4, are deployed on three machines, i.e., Machines 1–3. Each service has multiple container instances, totaling N container instances. We found that S_1 has a slow response time and would like to determine the root cause. We start by extracting metrics related to containers from the metric layer, such as container CPU usage and memory usage. These metrics are then fed into our proposed transformer-based anomaly detector (TAD), which employs multiple transformer encoder modules to reconstruct the input metrics, generating anomaly scores s_n for container n. Containers with top-k anomaly scores are identified as the initial faulty area in the resource layer. We also analyze the traces in the service layer using BFS and KDE, disassembling remote procedure call (RPC) latency and discarding downstream redundant call latency in order to focus on analyzing the latency of each service, analyzing the root cause service as S_2. We combine the resources associated with this service with the anomalous containers detected by the TAD to determine the final faulty area. We then collect metrics from the metrics layer for this failure area and evaluate them based on multiple factors to obtain a ranking of the root cause metrics. The rankings show that the most likely root cause is high CPU usage on machine 1. If only the root cause service S_1 is detected like other methods, operations personnel will still need to start from scratch to determine whether the issue lies with the service itself or with the underlying resources. However, our approach can provide them with a more precise target, leading to clearer solutions, such as reducing CPU usage or migrating the service.

IV. Anomaly Detection

In this section, we begin by presenting the problem definition. We subsequently introduce the key components of TAD: the temporal block and container block, addressing metric temporal relationships and container relationships, respectively. Then, we provide an overview of the structure of the model, loss function, and anomaly detection method.

A. Problem Definition

Notations used for anomaly detection are listed with descriptions in Nomenclature. We define that there are N containers in a microservice system, and we collect M metrics from each container for a total of T minutes to get $\mathcal{T} = \{x^1, \ldots, x^T\} \in \mathbb{R}^{N\times M\times T}$, where the data at minute t are:

$$x^t = \begin{bmatrix} x^t_{(1,1)} & \cdots & x^t_{(1,M)} \\ \cdots & \cdots & \cdots \\ x^t_{(N,1)} & \cdots & x^t_{(N,M)} \end{bmatrix}. \tag{1}$$

Let the length of the time window be L, the data for the ith window are defined as $X^i = \{x^{(i-1)*L+1}, \ldots, x^{i*L}\}$, and we get a total of T/L windows of data $X = \{X^1, \ldots, X^{T/L}\}$. We now define the two problems of microserivce anomaly detection and anomalous container localization.

1) *Microservice Anomaly Detection:* Given a training set of time-series data X collected from a microservice system, for a test set of window length L and time length $\hat{T}(\hat{T} \geq L)$ collected from the same system, predict $Y = \{y^1, \ldots, y^{\hat{T}/L}\}$, where $y^i \in \{0, 1\}$ represents whether microservice has an anomaly in the ith time window (1 denotes an anomalous window).
2) *Anomalous Container Localization:* Given the above training and test sets, predict $Y = \{y^1, \ldots, y^{\hat{T}/L}\}$, where $y^i_{(n,\cdot)}$ indicates whether the nth container has an anomaly in the ith time window.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

B. Temporal Block

Temporal block primarily comprises two components: temporal attention module and positionwise feed-forward module.

1) Temporal Attention: Vanilla self-attention is used here to capture temporal relationships, but there has been a lot of work [23], [24], [25], [26] on improving the efficiency and performance of attention, and attention here should be an open and replaceable module that can be chosen according to the usage scenario. First, we expand the data of each time window X^i from X into a 2-D matrix $\mathcal{X} \in \mathbb{R}^{D\times L}$ with timestamps as columns, where $D = N \times M$. The expanded format is as follows:

$$\mathcal{X} = \begin{bmatrix} x_{(1,1)}^{(i-1)*L+1} & \cdots & x_{(1,1)}^{i*L} \\ \cdots & \cdots & \cdots \\ x_{(1,M)}^{(i-1)*L+1} & \cdots & x_{(1,M)}^{i*L} \\ \cdots & \cdots & \cdots \\ x_{(N,1)}^{(i-1)*L+1} & \cdots & x_{(N,1)}^{i*L} \\ \cdots & \cdots & \cdots \\ x_{(N,M)}^{(i-1)*L+1} & \cdots & x_{(N,M)}^{i*L} \end{bmatrix}. \tag{2}$$

Temporal attention (TA) employs multihead attention. Each attention head can focus on different segments of the data and generate distinct attention weight distributions. The outputs from these multiple attention heads are combined, thereby representing various aspects of the data in higher dimensions. This enhances the model's capacity for expression and generalization. TA can be expressed as

$$\begin{aligned} \mathrm{TA}(\mathcal{X}) &= \mathrm{Concat}(\mathrm{head}_1, \mathrm{head}_2, \ldots, \mathrm{head}_H) \cdot W_0 \\ \mathrm{head}_h &= \mathrm{Attention}(Q_h, K_h, V_h) \end{aligned} \tag{3}$$

where $Q_h = W_q^h \mathcal{X}$, $K_h = W_k^h \mathcal{X}$, $V_h = W_v^h \mathcal{X}$, and H is the number of heads. Vanilla self-attention here can be expressed as

$$\mathrm{Attention}(Q, K, V) = \mathrm{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right) V. \tag{4}$$

2) Positionwise Feed-Forward Module: This module primarily involves two linear transformations and an intermediate ReLU activation to facilitate feature fusion and transformation

$$\mathrm{FFN}(Z) = \mathrm{ReLU}(Z * W_1 + b_1)W_2 + b_2 \tag{5}$$

where Z is the input to the module and W_1 and W_2 are the weights of the linear transformation.

To enhance efficiency, the skip connection and layer normalization are introduced after the operation of the two modules. Consequently, the temporal block can be represented as

$$\begin{aligned} Z &= \mathrm{Norm}(\mathrm{TA}(\mathcal{X}) + \mathcal{X}) \\ O &= \mathrm{Norm}(Z + FFN(Z)) \end{aligned} \tag{6}$$

where O is the output of temporal block, norm is layer normalization.

C. Container Block

The container block comprises a sole container attention module. In this article, we introduce an attention mechanism to capture the reciprocal influence and interactions among containers, termed as container self-attention. The specific structure is shown in Fig. 4. First, we transform the output $O \in \mathbb{R}^{D\times L}$ of temporal block into $R \in \mathbb{R}^{N\times M\times L}$. Subsequently, the data from each container R_n is forwarded through a straightforward fully connected layer, generating embeddings for each container. These embeddings are concatenated to form the query matrix (Q) and key matrix (K), structured as follows:

$$\begin{aligned} q_n &= f\left(W_q^n * R_n + b_q^n\right), k_n = f\left(W_k^n * R_n + b_k^n\right) \\ Q &= \mathrm{Concat}\left(q_1, \ldots q_N\right), K = \mathrm{Concat}\left(k_1, \ldots k_N\right) \end{aligned} \tag{7}$$

where q_n, k_n represent the embedding vector of container n. Subsequently, Q and K matrix are employed to compute the correlation matrix of attention

$$A = \mathrm{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right). \tag{8}$$

For the output $\tilde{X} \in \mathbb{R}^{N\times M\times T}$, the data of container n are expressed as

$$\tilde{x}_n = \sum_{j=1}^{N} a_{n,j} * R_n \tag{9}$$

where $a_{n,j}$ is the element in the nth row and jth column of matrix A.

D. Overall Structure of TAD

The overall structure is shown in Fig. 3. Before inputting the data, we add positional embedding which is used to add localization information to the time-series dimension. The data are fed into an encoder consisting of three TAD blocks, where the temporal block learns the correlation between the time series through a multihead attention mechanism. Meanwhile, the container block models the relationships between container-based time-series data, capturing the relationships between containers. The information between the two modules is then fused and propagated through a sandwich structure inspired by the ST-Conv module in STGCN [27]. Following the decoding of the reconstructed time-series data, both the reconstructed and actual data are fed into the scorer to calculate the anomaly score.

E. Loss Function

For TAD, taking into account potential significant fluctuations in the data, we use smooth in the objective function. The technique was initially introduced in fast-RCNN [28] to reduce the interference of outliers on gradients, thereby enhancing the robustness of training. Its form is

$$\mathrm{smooth}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise.} \end{cases} \tag{10}$$

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

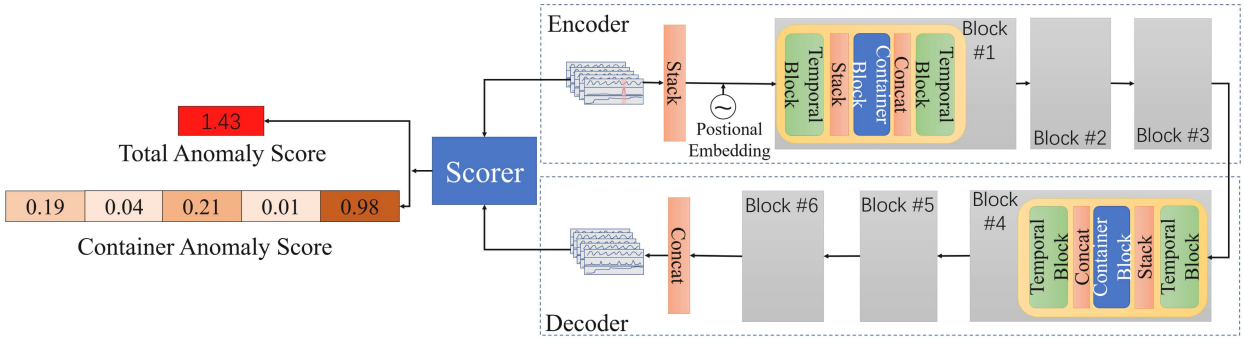


Fig. 3. Architecture of proposed anomaly detection model TAD.

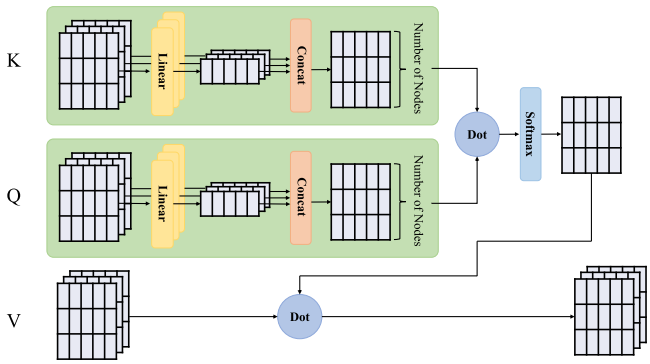


Fig. 4. Structure of container self-attention.The green part shows the Q and K parts, the process of embedding each node from the FC layer to the container node.

Our objective function is

$$\mathcal{L} = \sum_{n=1}^{N}\sum_{m=1}^{M}\sum_{t=(i-1)*L+1}^{i*L} \text{smooth}\left(x_{(n,m)}^{t} - \hat{x}_{(n,m)}^{t}\right) \quad (11)$$

where x is input data and $\hat{x}$ is reconstructed data.

F. Anomaly Score

The model is trained on normal data, with the primary objective being the reconstruction of these normal data points. In the real-life scenarios with anomalies, the model relies on its learned patterns to reconstruct data, thereby enabling anomaly detection. During the training phase, the maximum value of the mean squared error between input data and reconstructed data is employed as the anomaly threshold for detection. In the detection phase, an anomaly is flagged if the reconstruction error surpasses the threshold. The reconstruction error, also referred to as the total anomaly score, is formulated as follows:

$$\mathrm{s} = \sum_{n=1}^{N}\sum_{m=1}^{M}\sum_{t=(i-1)*L+1}^{i*L}\left(x_{(n,m)}^{t} - \hat{x}_{(n,m)}^{t}\right)^2 \quad (12)$$

$$\mathrm{s}_n = \sum_{m=1}^{M}\sum_{t=(i-1)*L+1}^{i*L}\left(x_{(n,m)}^{t} - \hat{x}_{(n,m)}^{t}\right)^2. \quad (13)$$

When anomalous data are present in the input, the reconstruction errors of each container s_n, also called container anomaly scores are arranged in descending order, and the top-k containers with the highest scores are identified as the anomalous containers.

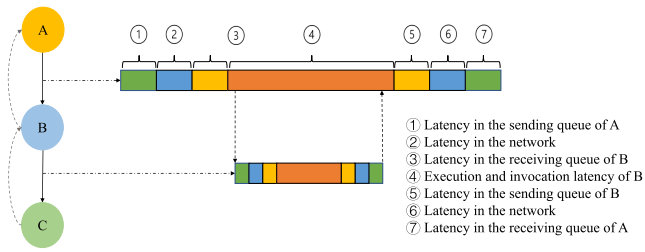


Fig. 5. Detailed breakdown of RPC latency, where the green section represents the client queuing delay, the blue section represents the network latency, the yellow section represents the server queuing delay, and the orange section represents the runtime of server.

V. Faulty Area Discovery

After obtaining the collection of anomalous containers through the TAD model, this section further determines the faulty area by discomposing and analyzing the RPCs of traces in the service layer. This section first introduces the concept of differential invocation latency, which can help us focus on detecting each service without being affected by downstream redundant calls. Then we propose to use a service anomaly detector using KDE, and finally, using this existing information, we propose a BFS-based faulty area detection algorithm.

A. Differential Invocation Latency

Due to the dependencies between services, faulty services can be located and potential faulty areas can be identified by analyzing traces. Specifically, as shown in Fig. 5, we break down and analyze the RPC latency in traces.

The entire invocation process is divided into 7 parts. As RPC in microservices generally communicates based on TCP, processes need to push data into the sending queue first, and then the network interface card (NIC) sends it to the network. Similarly, when receiving, the NIC also first receives the data into the receiving queue. When the queue meets certain conditions (for example being full), the information in the receiving queue will be delivered to the upstream process. Therefore, in RPC request and response, there are delays in the queue, and the ①;③;⑤;⑦ part in the figure represents these delays. During the transmission

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

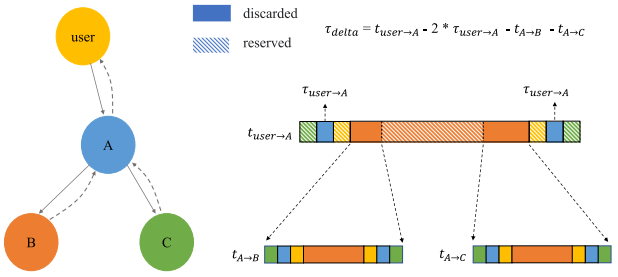


Fig. 6. Example of differential invocation latency.

of the message in the network, there is also a latency represented by the ②;⑥ part in the figure. To obtain precise network latency, we need tools like SystemTap [29] to probe Linux kernel with a notable performance overhead. Inspired by the Sage [30], the latency of heartbeat signals is measured to approximate network latency. The other parts are the processes of the RPC receiver calculating the result and preparing to return it, including the serialization and deserialization of the RPC. By splitting these parts, this article can locate the faulty area more accurately in the inspection of traces.

After analysis, it can be concluded that there are three possible reasons for problems exhibited by RPC call, for example, service A calls server B: 1) issues with the sending or receiving queues of the container or physical machine where service A is located; 2) network communication issues between containers or physical machines; 3) issues with service B. Since the latency of each invocation is very complex, especially the latency of the current call may include the overhead of all downstream calls, this study proposes the concept of differential invocation latency to simplify the latency of traces.

Assuming there is a trace as shown in Fig. 6, if the invocation latencies for user to A, A to B, and A to C are $t_{\text{user}->A}$, $t_{A->B}$, and $t_{A->C}$, respectively, the average network latency for user to A is $\tau_{\text{user}->A}$, then for service A, the differential invocation latency is $\tau_{\text{delta}} = t_{\text{user}->A} - 2 * \tau_{\text{user}->A} - t_{A->B} - t_{A->C}$. Therefore, for service A, its differential invocation latency represents the queuing delay of the machine where the service user is located and the service time of A itself on the current machine. This method separates a call from its downstream calls, which facilitates more accurate anomaly detection for each service.

The differential invocation latency is formally defined as

$$\tau_{\text{delta}} = t_{\text{prev}->\text{cur}} - 2 * \tau_{\text{prev}->\text{cur}} - \sum_{i=0}^{n} t_{\text{curr}->i} \tag{14}$$

where $t_{\text{prev}->\text{cur}}$ represents the invocation latency when the upstream service invokes this service, $t_{\text{cur}->i}$ represents the latency for calling other services by the current service, $\tau_{\text{prev}->\text{cur}}$ is the network latency when the upstream service calls current service, and n is the number of services called by the current service.

B. Anomaly Detector Using KDE

With the differential invocation latency, we can perform anomaly detection for each service node, which lays the foundation for root cause analysis.

As the invocation latency does not change over time, there is almost no autocorrelation in the temporal order, so we use a probabilistic method to detect anomalies. In contrast to assuming a specific distribution, such as a normal distribution, we believe that refraining from making explicit assumptions about the distribution of the data and directly constructing an estimation function from the sample data through nonparametric estimation is more robust and accurate. In this study, we use KDE to estimate the distribution of differential invocation latency under normal conditions for different nodes. KDE is a typical nonparametric estimation method, which estimates the probability density function (PDF) $\hat{f}(x)$ without any assumptions by superimposing several kernel functions

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right). \tag{15}$$

Here, K represents the kernel function, h represents the smoothing parameter, and n represents the number of observation points. Commonly used kernel functions include Gaussian, tophat, linear, and so on. In this study, we use the Gaussian kernel function

$$K(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}x^2\right). \tag{16}$$

After applying KDE to the stable data, we obtain the probability distribution function of the differential invocation latency for that node, and determine its upper quartile $\widehat{x_\alpha}$ and lower quartile $\widetilde{x_\alpha}$

$$\alpha = \int_{\widehat{x_\alpha}}^{+\infty} \hat{f}(x)dx \tag{17}$$

$$\alpha = \int_{-\infty}^{\widetilde{x_\alpha}} \hat{f}(x)dx. \tag{18}$$

Similar to the three-sigma rule for normal distribution, when α is small enough, if it falls into the range of $(-\infty, \widetilde{x_\alpha})$ or $(\widehat{x_\alpha}, +\infty)$, it should be considered as an outlier event.

C. Root Cause Area Discovery

With the anomaly detector using the KDE above, it is possible to locate the suspicious faulty area. Based on the observation of various traces in the preliminary dataset of the AIOps2020 International Challenge, we summarize the characteristics of the root cause area as follows:

Observation 1: Faults often propagate from bottom to top. For example, when a database times out, the running status of all upstream invocations is affected to varying degrees. Although the differential invocation latency eliminates some of the effects of upstream and downstream calls, anomalies still cause varying degrees of fluctuation in the service's runtime.

Observation 2: Different services on the same machine or container may also experience certain faults when one service fails, but the number of faults and the start time of the faults are delayed relative to the container or machine associated with the root cause.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: BFS-Based Algorithm for Discovering Potential Root Cause Area.

Require: Q, the queue stores all trace trees of the anomalous interval; $Detector$, the detector for differential invocation latency of traces using KDE; CS_{TAD}, anomalous containers output by TAD; d_m, maximum depth of all trace trees

1: $M_C \leftarrow$ initial the mapping from node to its access count
2: $M_A \leftarrow$ initial the mapping from node to its anomalous count
3: **repeat**
4: $T \leftarrow$ pop a trace tree from Q
5: $NodeQ \leftarrow$ initial the node queue
6: push root node v_r of T to the end of $NodeQ$
7: **repeat**
8: $v \leftarrow$ pop node from $NodeQ$
9: $M_C[v] \mathrel{+}= 1$
10: **if** $Detector(Q)$ **then**
11: $M_A[v] \mathrel{+}= 1$
12: **end if**
13: **for** each child node v_c of v in T **do**
14: push v_c to the end of $NodeQ$
15: **end for**
16: **until** $len(NodeQ) = 0$
17: **until** $len(Q) = 0$
18: $s_m \leftarrow 0$
19: **for** v, c in M_A **do**
20: $c_m \leftarrow M_C[v]$
21: $s \leftarrow c/c_m + \lambda * depth(v)/d_m$
22: **if** $s > s_m$ **then**
23: $s_m \leftarrow s$
24: $v' \leftarrow v$
25: **end if**
26: **end for**
27: $v'_p \leftarrow$ parent service node of root cause service node v'
28: $CS_{v'} \leftarrow$ containers of v' and v'_p
29: $CS_r \leftarrow$ intersection of $CS_{v'}$ and CS_{TAD}, final potential root cause containers
30: $PA \leftarrow$ potential root cause area consisting of CS_r, v', and v'_p
31: **return** PA

Based on these two observations, the localization method is summarized in Algorithm 1. We use BFS to traverse each trace tree, count the number of occurrences of each service node as well as the number of times it is judged as anomalous (from Line 9 to Line 12), and score them based on the node's abnormality count and depth (Line 21, λ is a hyperparameter for balance). Then, the service node with the highest score is considered as the root cause node. Finally, we add the node and its parent service node to the potential root cause area, along with the intersection CS_r of all containers in these two nodes and anomalous containers detected by TAD.

VI. Root Cause Metric Localization

After identifying the faulty area, the next step is to collect metrics from these containers or machines to determine the root cause of the failure. After analyzing the data of AIOps2020 competition, this study found the following phenomena for the root cause metric. Observation 1: The root cause fluctuations are always related to differential invocation latencies. Observation 2: The root cause metric is often the most volatile metric. Observation 3: The root cause metric undergoes a mutation earlier than any other changing metrics.

Based on the three observations above, assuming that each metric is a time series o, we calculate the scores of them by combining the three characteristics.

Based on observation 1, the data are first subtracted from its average value during the stable period, which can alleviate the influence on calculating the causality

$$\tilde{o} = o - o_m. \tag{19}$$

The variable o_m represents the mean value of the sequence under stable and normal conditions. Then, a Granger causality test is performed between each metric and the differential invocation latencies in the trace. This test is used to determine whether there is a causal relationship between two time series, with a smaller p-value indicating a stronger causal relationship. Therefore, this study defines the causal coefficient of all metrics as $s_{\text{casual}} = 1 - p$, where $p \in [0, 1]$.

We evaluate the fluctuation in observation 2 by calculating the distance between the distributions of the metric in the absence of faults and in the presence of faults. As different metrics have different dimensionalities, we first normalize the data using the maximum and minimum values, and denote the processed sequence as $\dot{o}$. The difference in data distribution is defined as

$$s_{\text{fluct}} = WassersteinDistance(\dot{o}_{\text{norm}}, \dot{o}_{\text{fault}}) \tag{20}$$

where $\dot{o}_{\text{norm}}$ is the normal time series, $\dot{o}_{\text{fault}}$ is the anomalous time series, and $s_{\text{fluct}} \in [0, 1]$

According to observation 3, metrics that fluctuate earlier are more likely to be the root causes. We identify the start time of the fluctuation by calculating the absolute derivative value of each metric data point, which is designed by [31]. Let the time of the inflection point be t'

$$t' = \text{argmax}(|o_{\text{prev}} - o_{\text{next}}|) \tag{21}$$

where o_{prev} and o_{next} are two adjacent points, and t' is the time point corresponding to the maximum change of the metric. Time score s_t then can be written as

$$s_t = \frac{|t - t'|}{\Delta t_{\text{max}}}. \tag{22}$$

Here, t is the time point of the detection, Δt_{max} is the maximum value of the time difference between t and each t', $s_t \in [0, 1]$.

Therefore, this study combines the scores representing causality s_{causal}, fluctuation s_{fluct}, and time distance of change s_t to propose a multifactor root cause score s_{root}

$$s_{\text{root}} = \alpha \times s_{\text{causal}} + \beta \times s_{\text{fluct}} + \gamma \times s_t \tag{23}$$

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TYPES AND QUANTITIES OF FAULTS IN THE AIOPS 2020 PRELIMINARY DATASET

Fault type	Quantity
CPU fault	19
DB connection close	5
DB connection limit	7
Network delay	31
Network loss	19

where $\alpha + \beta + \gamma = 1$, $s_{\text{root}} \in [0, 1]$. If we calculate the scores of all metrics in the collection S, then the ranking of the root cause is determined by the ranking of the scores in S

$$RM_{\text{LIST}} = \text{top-k}(S_{\text{score}}) \tag{24}$$

where RM_{LIST} is the top k root cause metrics.

VII. EXPERIMENTS

Within this section, our primary focus revolves around the introduction of datasets, the hyperparameters of the models, the performance of the models, and a comprehensive comparison with baseline models.

We use SMD,[5] AIOps2020 Preliminary Dataset, [6] and Sock-Shop Dataset described as follows.

SMD: A dataset spans a duration of five weeks and originates from a prominent Internet corporation, which was published with the OmniAnomaly [20] model.

AIOps2020 Preliminary Dataset: It is collected from a microservices system in a testing environment of a large operator. Its microservices are also containerized to support their basic services and communicated with each other through RPC. There were 81 anomalous cases in this dataset, including the injected anomalies shown in Table I, and each fault injection time lasted for 5 min. The whole data were simulated for 22 days.

Sock-Shop Dataset: Sock-Shop[7] is a microservices demo and we deploy it within a Kubernetes cluster with three nodes. Each service, excluding the database, has two pod replicas, and the resource provisioning is set to default configurations. We use Locust[8] to develop a workload generator and randomly inject faults using chaosblade.[9] We use Node exporter and cAvisor for monitoring and we collect data through Prometheus for 7 days. There are 102 simulated faults injected with a minimum interval of 30 min between injections. The injected faults lasted for 5 min each and there were four types of faults injected, as shown in Table II.

A. Evaluation Metrics

There are five evaluation metrics used in this article for anomaly detection: precision, recall, $F1$, area under the ROC curve (AUC), and average precision (AP). Precision quantifies the ratio of true positives among predicted positives, while recall represents the proportion of true positives within actual positives. They can be calculated as follows:

$$P = \frac{TP}{TP + FP}, R = \frac{TP}{TP + FN} \tag{25}$$

where TP is the number of true positive samples, FP is the number of false positive samples, and FN is the number of false negative samples. And P denotes the precision rate, R denotes the recall rate.

$F1$ denotes the harmonic average of the precision and recall values of the model's prediction outcomes. Its formula is

$$F_1 = \frac{2}{R^{-1} + P^{-1}} = 2 \cdot \frac{P \cdot R}{P + R}. \tag{26}$$

AUC stands for area under the ROC (receiver operating characteristic) curve and is commonly utilized to assess the performance of binary classifiers. A high AUC value indicates that the classifier has good performance and is able to effectively differentiate between positive and negative samples. The ROC curve represents the classifier's true positive rate (recall) versus false positive rate at different thresholds.

AP finds the area under the precision-recall curve, which is often used to evaluate the effect of classification models.

We use precision at k ($PR@k$), mean average precision (MAP) and $F1$ to evaluate the performance of root cause localization. $PR@k$ refers to the precision when the top k ranked root causes are considered as the actual root cause

$$PR@k = \frac{1}{|A|} \sum_{a \in A} \frac{\sum_{i<k} (R[i] \in v_{rc})}{(\min(k, |v_{rc}|))} \tag{27}$$

where v_{rc} is the root cause set, $|v_{rc}|$ is the size of the root cause set, $R[i]$ is the ith root cause, A is the fault set, and $|A|$ is the number of faults.

MAP mainly measures the effectiveness of the entire model

$$MAP = \frac{1}{|A|} \sum_{a \in A} \sum_{1 \le k \le N} PR@k \tag{28}$$

where N is the number of selected metrics.

B. Experiment Setup

1) Anomaly Detection: For baselines that only consider temporal relationships, we do the same as the TopoMAD [11] approach, combining all the metrics from the different components into one huge vector and using that vector as input. For classic models, we use scikit-learn library [32] with default parameters, and for the other models, we use the hyperparameters given in this article and implement them using PyTorch [33] on NVIDIA

TABLE II
TYPES AND QUANTITIES OF FAULTS INJECETED IN SOCK-SHOP

Fault type	Quantity
CPU fault	30
Network queue fault	25
database fault	26
Memory lack	21

[5][Online]. Available: https://github.com/NetManAIOps/OmniAnomaly
[6][Online]. Available: https://github.com/NetManAIOps/AIOps-Challenge-2020-Data
[7][Online]. Available: https://github.com/microservices-demo/microservices-demo
[8][Online]. Available: https://locust.io
[9][Online]. Available: https://github.com/chaosblade-io/chaosblade

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

Method	Sock-shop DataSet			SMD		
	Precision	Recall	F1	Precision	Recall	F1
Classical statistical Model						
LocalOutlierFactor	0.8214	**0.9415**	0.8773	0.5312	0.8103	0.6417
One-Class SVMs	0.8135	0.9365	0.8706	0.3559	0.8662	0.5045
Forecasting based models						
DAGMM	0.8988	0.8194	0.8572	0.5713	0.8603	0.6866
Reconstruction-based Model						
TopoMAD	0.8813	0.8001	0.8387	0.7013	0.9003	0.7884
OmniAnomaly	0.9125	0.8019	0.8536	0.8126	0.9503	0.8760
TAD	**0.9474**	0.8422	**0.8917**	**0.8334**	**0.9581**	**0.8914**

The bold values given the metrics, the score corresponding to the best performing model.

Fig. 7. Performance of our detection model TAD and baselines. Given the metrics, the score corresponding to the best performing model.

RTX3090 GPU. Our model is optimized using the Adam optimizer with an initial learning rate of 0.001. The input data is processed in batches of size 100, with a time window length of 5, and the number of stacks of the model is 3. We choose hyperparameters other than time window size using grid search, and the effect of window size on the performance of anomaly detection is discussed in Section VII-H.

2) Root Cause Localization: For baselines, we use the same setup as for MicroRCA [12], where the pseudoanomaly clustering algorithm is not implemented for MonitorRank [21] to identify the external factors since we only consider internal factors as in MicroRCA. Since all these methods localize only to the service layer, for comparison purposes, our method is used to localize to the root cause metrics after localizing to the service layer. The KDE algorithm we proposed is also implemented using scikit-learn library [32]. λ in the potential root cause area discovery algorithm is 1.5. α, β, and γ in the multifactor root cause score is 0.23, 0.55, and 0.22. The hyperparameters of our proposed root cause localization method are also determined by grid search.

C. Data Preprocessing

To mitigate dimensionality effects, data normalization are conducted prior to training. For the time series of the ith dimension denoted as x^i, normalization is executed using the maximum and minimum values from the training data

$$\widetilde{x^i} = \frac{x^i - \min\left(X^i_{\text{train}}\right)}{\max\left(X^i_{\text{train}}\right) - \min\left(X^i_{\text{train}}\right)}. \tag{29}$$

D. Overall Performance of Anomaly Detection

In this section, we will compare the performance of our model with other baseline models for anomaly detection using the Sock-Shop dataset and SMD. Fig. 7 presents the precision, recall, and F1 score of models. Evidently, our model outperforms or matches the baseline anomaly detectors. We also draw the ROC curve and P-R curve (see Fig. 8). It can be seen that in the two datasets, TAD is the closest to the top left corner in the ROC figure and to the top right corner in the P-R figure, and the area surrounded by the curve in the two figures is the largest, which indicates that the effect of TAD is better than that of the baseline model without considering the specific threshold setting and only considering the overall performance of the model.

In our experiments, classic methods like LOF and one-class SVM exhibit comparatively poorer performance due to the insufficient complexity.

DAGMM faces two primary issues. First, it overlooks the historical dependence within a time series, instead focusing solely on multidistribution modeling of the data. Second, it models the value of a single time point as a normal distribution, which fails to fit numerous special cases.

The drawback of OmniAnomaly is that it cannot clearly address feature correlation within the model. Although TopoMAD incorporates static topology graph information, its enhancement in model performance is restricted due to the dynamic interactions commonly observed among containers.

In contrast to the aforementioned models, our proposed approach employs the attention mechanism to model the dynamic influence between containers. Additionally, the temporal block captures temporal dependencies within time series. We integrate the spatial and temporal information between containers to achieve state-of-the-art performance.

E. Performance of Anomaly Detection Latency

In real-world microservices scenarios, the ability to promptly detect anomalies holds significant importance. Referring to SR-CNN [34], we calculate the average latency l of anomaly detection by models as follows:

$$l = \sum_{i \in \mathcal{A}} \frac{l_i}{|\mathcal{A}|} \tag{30}$$

where $\mathcal{A}$ represents the set of anomaly blocks, l_i signifies the minimum distance in terms of time steps between the anomaly detection point in the ith anomaly block and the point at which the anomaly occurs as shown in Fig. 9. Fig. 10 illustrates a comparison of the latency metric between our model and baseline models on Sock-Shop dataset and SMD. Evidently, TAD exhibits the lowest latency in detecting anomalies in the Sock-Shop dataset characterized by relatively brief anomaly duration and in SMD, which features prolonged anomaly duration. As a result,

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

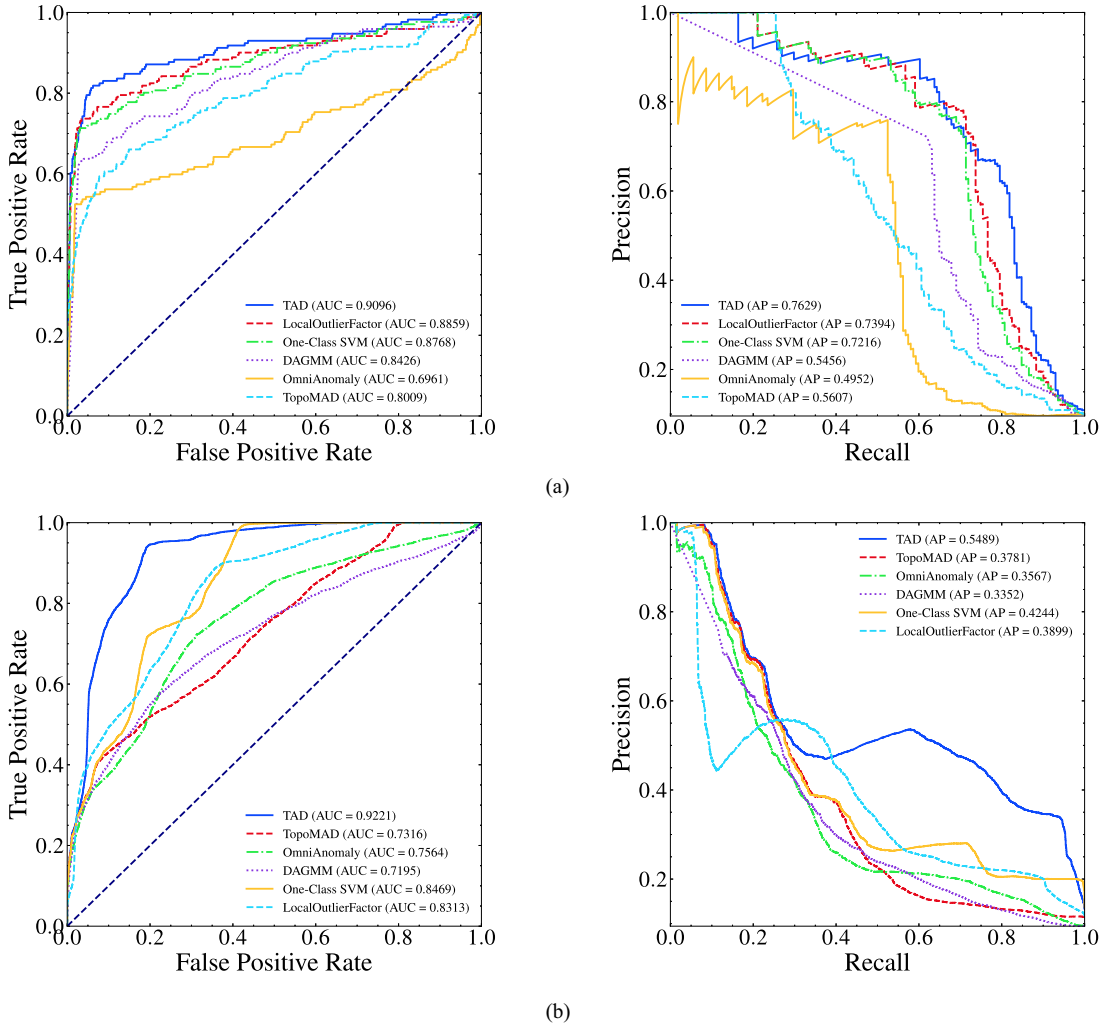


Fig. 8. ROC and R-P curves of different models on (a) the Sock-Shop Dataset and (b) SMD. The solid blue line is TAD, which is better than other baseline methods in both the area under the line and the degree close to the upper left corner. (a) Sock-Shop Dataset. (b) SMD.

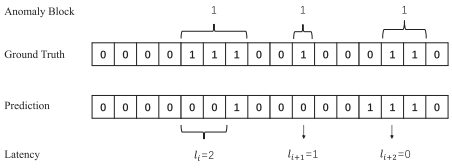


Fig. 9. Latency of detecting anomalies.

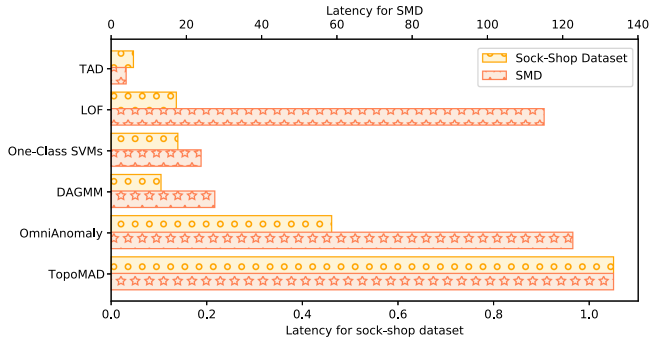


Fig. 10. The latency comparison between TAD and the baseline models on the Sock-Shop Dataset and SMD. The lower x-axis corresponds to the Sock-Shop Dataset, while the upper x-axis corresponds to SMD.

our TAD model is proficient in swiftly identifying anomalies in real-world scenarios, thus reducing the time required for root cause localization.

F. Overall Performance of Root Cause Localization

In order to verify the reliability of the proposed model, this section will compare the performance of it with three other baseline algorithms: MonitorRank, Microscope, and MicroRCA. We use Sock-Shop dataset and AIOps2022 preliminary dataset here, which is slightly different from anomaly detection models, to better demonstrate the wide applicability of our model.

Table III shows the results of baseline algorithms and the multilayered root cause localization method we proposed on

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PERFORMANCE COMPARISON OF OUR LOCALIZATION METHOD WITH BASELINES

Method	AIOps 2020 Preliminary Dataset				Sock-Shop Dataset			
	F1	MAP	PR@1	PR@3	F1	MAP	PR@1	PR@3
MonitorRank	0.22	0.41	0.24	0.37	0.32	0.70	0.38	0.66
Microscope	0.53	0.75	0.61	0.74	0.78	0.77	0.74	0.89
MicroRCA	0.61	0.92	0.86	0.93	0.87	0.91	0.87	0.92
OurMethod	**0.92**	**0.95**	**0.86**	**0.96**	**0.90**	**0.92**	**0.91**	**0.93**

The bold values given the metrics, the score corresponding to the best performing model.

the two datasets. It can be seen that the performance of the multilayered root cause localization algorithm far exceeds that of other models. Compared with other models, the multilayered root cause localization method has significant improvements in all evaluation metrics.

MonitorRank proposes a random walk algorithm by simulating the operation and maintenance personnel. However, the custom weight of edges does not consider the delay propagation of faults, resulting in a large number of misjudgments and reporting a large number of normal metrics and nodes.

Microscope uses the PC algorithm to construct the casual relationship of services, but the PC algorithm can only calculate the instantaneous causal graph, while the influence between metrics is a process that requires a certain time delay. Second, it uses the Pearson correlation coefficient to calculate the correlation between anomalous front-end service metrics and root cause candidate metrics. for a mutant metric, its correlation with the front end is very low, and the Pearson correlation coefficient mainly considers linear correlation, while the relationship between root causes and front-end metrics is often not linearly related. Therefore, many other metrics that grow with increased service will be considered root causes, more typically memory usage.

MicroRCA uses an attribute graph to construct the overall relationship graph and determines the root cause through random walk. When randomly walking, it still uses correlation as the weight, so it also faces the same problem as a microscope. However, its attribute graph adds the relationship between containers and physical machines, so its effect is slightly better than that of a microscope.

Unlike other methods, the multilayered root cause localization method determines the potential root cause area through differential latency and the BFS-based faulty area discovery algorithm. The prior operation and maintenance experience are incorporated into the algorithm, which improves the interpretability and effectiveness of the algorithm.

G. Ablation Study of Anomaly Detection Model

The main goal of this part is to verify the reliability of each part of TAD, and the main verification is the effect of modeling the time series and the relationship between the containers. Since the Sock-shop dataset is more in line with the characteristics of microservices, this part mainly uses this data as the experimental dataset. To study each component of our approach's effectiveness, we first removed the container block of the model and performed experiments to get the experimental results, and then replaced all temporal blocks with container blocks and then performed the experiment again. The experimental results obtained are shown in Table IV.

TABLE IV
ABLATION STUDY OF TAD

	Our model	w/o Container block	w/o Temporal block
F1	0.8572	0.8322	0.7287
AP	0.7629	0.72542	0.5413
AUC	0.9096	0.8177	0.8011

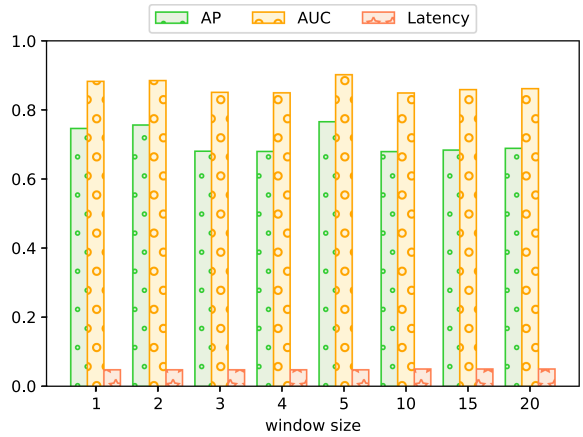


Fig. 11. Changes of the three indicators of AP, AUC, and latency on the Sock-Shop dataset of the TAD model as the window size changes.

It can be seen that after removing the container block, the model effect decreases slightly, and after removing the temporal block alone, the model effect decreases significantly. The main reason is that the dynamic relationship between the containers in the microservice system has little effect on the performance parameters of each node, especially under normal conditions, and the performance indicators are all time-series data. If you remove the time-series modeling, the effect of the model will be greatly reduced. When the two modules are combined, the dynamic relationship and timing characteristics of the container are learned, so the effect is the best.

H. Impact of Time Window

This section mainly discusses the impact of the time window size on the overall effect of the TAD model. In the experiment, we increase the time window size from 1 to 20, and then record the effect of TAD on the sock-shop dataset. As shown in Fig. 11, as the time window increases, the model effect is first decreased, then increased, and then decreased, and does not change after

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

TABLE V
ABLATION STUDY OF MULTILAYERED ROOT CAUSE LOCALIZATION ALGORITHM

	Causality score only	Fluctuation score only	Time score only
F1	0.10	0.71	0.32
MAP	0.25	0.87	0.12

the time window reaches 10. When the time window is 1, the characteristics of the cycle time sequence are not used. At this time, the spatial relationship mainly captured by the model is less effective. As the time window increases, although TAD begins to capture some time-related information, because the time window does not reach a certain length, it cannot fully capture the periodic information, and the model effect at this time will even gradually deteriorate. When the time window reaches the right time, that is, $window_size = 5$, the model effect will be remarkably compiled. Later, as the time window of the model continues to increase, the difficulty of reconstruction is also increasing, so the effect will be worse.

I. Ablation Study of Root Cause Localization Algorithm

In this section, we try to verify the reliability of each part of multilayered root cause localization algorithm. We make some adjustments to the entire method, using the score of one specific factor as the root cause score in root cause analysis. Table V shows its effectiveness.

The results of the ablation experiment show that using only the fluctuation score model has the best performance, followed by the time score model, while using only the causality score model has the worst performance. This indicates that the dominant feature in locating the root cause is the severity of the fluctuations, but in some cases, there may be multiple options with severe fluctuations, making it difficult to determine the true root cause using the fluctuation score only. Similarly, if only the time score is used, small fluctuations in some metrics may be directly considered as the root cause, leading to many false positives. However, when locating container failures, the metrics changes are relatively stable, and directly ranking them as root cause scores can still obtain some correct results. The causality score is the worst because the Granger causality test used can only detect linear causality without hidden variables, hence its poor performance. However, for network issues, its metrics are generally directly proportional to latency. In this case, using causal relationships can help locate some of the root causes.

VIII. CONCLUSION

This article analyzed the entire microservices architecture and focused on solving the problems of anomaly detection and root cause localization for microservice systems. Through capturing the dependencies between containers and the temporal relationship within a time window, our approach outperformed other state-of-the-art models for container anomaly detection on both the Sock-Shop dataset and SMD. Notably, our model also exhibited superior speed in anomaly detection when compared to alternative approaches. A faulty area discovery algorithm based on BFS was also proposed. By decomposing the tracing chain, the differential invocation latency obtained could more accurately confirm the faulty service. A multifactor root cause score was proposed to rank metrics of potential faulty area, and finally possible root cause metrics were obtained. This method also performed best on Sock-Shop dataset and AIOps2020 preliminary dataset. In summary, our models improved the reliability of microservice systems through efficient and accurate anomaly detection and root cause localization. In our future work, we will consider which metrics better expose microservice failures and try to combine data from other modalities for anomaly detection and root cause localization.

REFERENCES

[1] N. Dragoni et al., "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*. Cham, Switzerland: Springer, 2017, pp. 195–216.
[2] M. Lv, J. Fan, W. Fan, and X. Jia, "Fault diagnosis based on subsystem structures of data center network BCube," *IEEE Trans. Rel.*, vol. 71, no. 2, pp. 963–972, Jun. 2022.
[3] S. Meng, L. Luo, X. Qiu, and Y. Dai, "Service-oriented reliability modeling and autonomous optimization of reliability for public cloud computing systems," *IEEE Trans. Rel.*, vol. 71, no. 2, pp. 527–538, Jun. 2022.
[4] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Trans. Softw. Eng.*, vol. 42, no. 8, pp. 707–740, Aug. 2016.
[5] M.-C. Chiang, C.-Y. Huang, C.-Y. Wu, and C.-Y. Tsai, "Analysis of a fault-tolerant framework for reliability prediction of service-oriented architecture systems," *IEEE Trans. Rel.*, vol. 70, no. 1, pp. 13–48, Mar. 2021.
[6] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM Int. Conf. Manage. Data*, 2000, pp. 93–104.
[7] Y. Chen, X. S. Zhou, and T. S. Huang, "One-class SVM for learning in image retrieval," in *Proc. IEEE Int. Conf. Image Process.*, 2001, pp. 34–37.
[8] A. He and X. Jin, "Deep variational autoencoder classifier for intelligent fault diagnosis adaptive to unseen fault categories," *IEEE Trans. Rel.*, vol. 70, no. 4, pp. 1581–1595, Dec. 2021.
[9] P. Qi, D. Li, and S.-K. Ng, "MAD-SGCN: Multivariate anomaly detection with self-learning graph convolutional networks," in *Proc. IEEE Int. Conf. Data Eng.*, 2022, pp. 1232–1244.
[10] P. Malhotra et al., "Long short term memory networks for anomaly detection in time series," *Esann*, vol. 2015, 2015, Art. no. 89.
[11] Z. He et al., "A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1705–1719, Apr. 2023.
[12] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, 2020, pp. 1–9.
[13] M. Li et al., "Causal inference-based root cause analysis for online service systems with intervention recognition," in *Proc. 28th ACM Int. Conf. Knowl. Discov. Data Mining*, 2022, pp. 3230–3240.
[14] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
[15] M. Cinque, R. D. Corte, and A. Pecchia, "Micro2vec: Anomaly detection in microservices systems by mining numeric representations of computer logs," *J. Netw. Comput. Appl.*, vol. 208, 2022, Art. no. 103515.
[16] L. Wang, N. Zhao, J. Chen, P. Li, W. Zhang, and K. Sui, "Root-cause metric location for microservice systems via log anomaly detection," in *Proc. IEEE Int. Conf. Web Serv.*, 2020, pp. 142–150.
[17] P. Liu et al., "Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks," in *Proc. IEEE Int. Symp. Softw. Rel. Eng.*, 2020, pp. 48–58.
[18] K. Zhang, C. Zhang, X. Peng, and C. Sha, "PUTraceAD: Trace anomaly detection with partial labels based on GNN and PU learning," in *Proc. IEEE Int. Symp. Softw. Rel. Eng.*, 2022, pp. 239–250.
[19] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2018.
[20] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.

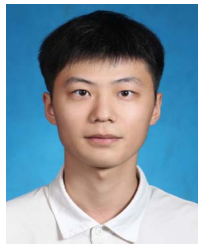
network," in *Proc. 25th ACM Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 2828–2837.
[21] M. Kim, R. Sumbaly, and S. Shah, "Root cause detection in a service-oriented architecture," in *Proc. ACM SIGMETRICS/Int. Conf. Meas. Model. Comput. Syst.*, 2013, pp. 93–104.
[22] J. Lin, P. Chen, and Z. Zheng, "Microscope: Pinpoint performance issues with causal graphs in micro-service environments," in *Proc. Int. Conf. Serv.-Oriented Comput.*, 2018, pp. 3–20.
[23] N. Kitaev, L. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," in *Proc. Int. Conf. Learn. Representations*, 2019.
[24] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11106–11115.
[25] S. Liu et al., "Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting," in *Proc. Int. Conf. Learn. Representations*, 2021.
[26] Y. Liu, H. Wu, J. Wang, and M. Long, "Non-stationary transformers: Exploring the stationarity in time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 9881–9893.
[27] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 3634–3640.
[28] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 1440–1448.
[29] P. Olivier, J. Boukhobza, M. Soula, M. L. Grand, I. C. Draa, and E. Senn, "A tracing toolset for embedded Linux flash file system," in *Proc. 8th Int. Conf. Perform. Eval. Methodol. Tools*, 2014, pp. 153–158.
[30] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, "Sage: Practical and scalable ML-driven performance debugging in microservices," in *Proc. ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2021, pp. 135–151.
[31] P. Liu et al., "FluxRank: A widely-deployable framework to automatically localizing root cause machines for software service failure mitigation," in *Proc. IEEE Int. Symp. Softw. Rel. Eng.*, 2019, pp. 35–46.
[32] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[33] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.
[34] H. Ren et al., "Time-series anomaly detection service at microsoft," in *Proc. ACM Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 3009–3017.



Jingyu Wang (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2008.

He is currently a Professor with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. He has authored or coauthored more than 200 papers in the JSAC, ToN, TPDS, NSDI, and so on. His research interests include broad aspects of intelligent networks, Edge/Cloud computing, machine learning, AIOps and self-driving network, IoV/IoT, knowledge-defined network and intent-driven networking.

Dr. Wang is Senior Member of CIC and selected for Beijing Young Talents Program.



Yuewei Li received the B.S. degree in computer science and technology from Xidian University, Xi'an, China, in 2018. He is currently working toward the M.S. degree in computer technology with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China.

His research interests include AIOps and cloud computing.



Qi Qi (Senior Member, IEEE) received the Ph.D. degree in computer science and technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2010.

She is currently a Professor with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. She has authored or coauthored more than 100 papers in international journals and conferences. Her research interests include time serials analysis, edge intelligent network, distributed machine learning, cloud computing, Internet of Things, and deep reinforcement learning.

Dr. Qi was the recipient of three awards from the National Natural Science Foundations of China.



Yan Lu received the M.S. degree in computer science and technology from the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China, in 2023.

His research interests include time series anomaly detection, root cause diagnose, and cloud computing.



Bo Wu received the master's degree in information and communicaton system from the Huazhong University of Science and Technology, Wuhan, China, in 2007.

He is the Group Manager and Deputy Director of the Department of Service Technology, China Mobile Research Institute, Beijing, China. His research interests include home bandwidth network service, future service for 6G, and AIOps for mobile networks and services.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:35:53 UTC from IEEE Xplore. Restrictions apply.